

WEEK-1**(Introduction to Java Basics and Class Fundamentals)**

1Q) Write a simple JAVA program that prints "Hello, World!" to the console.

PROCEDURE:

Create the Hello class with the main() method. Use System.out.println() to print the required message. Compile and run the program to display the message on the console.

System.out.println() statement is used to display the message "Hello World!" on the console. When the program is executed, the main() method runs and prints the output.

PROGRAM:

```
class Hello
{
    public static void main(String[] args)
    {
        System.out.println("Hello World!");
    }
}
```

OUTPUT:

Hello World!

INFERENCE:

The same output can also be displayed using System.out.print() instead of System.out.println().

Both methods print text to the console, but println() moves the cursor to the next line after printing.

2Q) Write a JAVA program to perform addition of two numbers using command line arguments.

PROCEDURE:

Create the main() method and receive the two values through args[]. Convert the arguments from String to integer values using Integer.parseInt(). Add the numbers and display the calculated sum.

accessed through the args[] array. The Integer.parseInt() method converts the string arguments into integers. The sum of the two numbers is calculated and displayed using System.out.println().

PROGRAM:

```
class Commandline
{
    public static void main(String[] args)
    {
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);
        int sum = a+b;
        System.out.println("Sum: " + sum);
    }
}
```

OUTPUT:

Sum: 50

INFERENCE:

The same addition can also be performed by taking input from the user using the Scanner class instead of command-line arguments. This method is more convenient for interactive programs.

3Q) Write a JAVA program to perform addition of two numbers using Scanner Class.**PROCEDURE:**

Import the Scanner class and create a Scanner object for keyboard input. Read two numbers from the user and store them in variables. Add the values and print their sum.

A Scanner object is created to read user input. The nextFloat() method reads two floating-point numbers, which are added together and displayed using System.out.println().

PROGRAM:

```
import java.util.Scanner;

class Sum
{
```

```
public static void main(String[] args)
{
    Scanner S = new Scanner(System.in);
    System.out.print("Enter First number: ");
    float num1 = S.nextFloat();
    System.out.print("Enter Second number: ");
    float num2 = S.nextFloat();
    float sum = num1+num2;
    System.out.println("Sum of two number: " + sum);
} }
```

OUTPUT:

Enter First number: 4.63

Enter Second number: 52.21

Sum of two number: 56.84

INFERENCE:

An alternative approach is to use command-line arguments with `Float.parseFloat()` instead of the `Scanner` class. Both methods can be used to accept input from the user.

4Q) Write a JAVA program to display default value of all primitive data types of JAVA.**PROCEDURE:**

Declare static variables for all primitive data types without assigning values. Access these variables from the `main()` method. Print each variable to observe the default value provided by Java.

`main()` method displays their default values using `System.out.println()`. Since the variables are declared as static and are not initialized, Java automatically assigns default values to them.

PROGRAM:

```
class Primitive
{
    static int a;
```

```
static float b;  
static char c;  
static double d;  
static boolean e;  
static long f;  
static short g;  
static byte h;  
  
public static void main(String[] args)  
{  
}  
}  
  
System.out.println("Int: " + a);  
System.out.println("Float: " + b);  
System.out.println("Char: " + c);  
System.out.println("Double: " + d);  
System.out.println("Boolean: " + e);  
System.out.println("Long: " + f);  
System.out.println("Short: " + g);  
System.out.println("Byte: " + h);
```

OUTPUT:

Int: 0

Float: 0.0

Char: ' '

Double: 0.0

Boolean: false

Long: 0

Short: 0

Byte: 0

INFERENCE:

The same concept can also be demonstrated by explicitly assigning values to each primitive variable before printing them. This helps compare default values with user-defined values.

5Q) Write a JAVA program that displays the roots of a quadratic equation $ax^2+bx+c=0$. Calculate the discriminant D and basing on the value of D , describe the nature of roots.

PROCEDURE:

Read the coefficients a , b and c using Scanner. Calculate the discriminant using $b*b - 4*a*c$ and check its value. Use the appropriate formula with `Math.sqrt()` to display the nature and values of the roots.

coefficients a , b , and c . The discriminant is calculated using the formula $d = b*b - 4*a*c$. An if-else statement checks whether the roots are real and distinct, real and equal, or complex. The `Math.sqrt()` method is used to calculate the square root for finding the roots.

PROGRAM:

```
import java.util.Scanner;

class Quadratic
{
    public static void main(String[] args)
    {
        Scanner S = new Scanner(System.in);

        System.out.print("Enter ax^2 term: ");

        int a = S.nextInt();

        System.out.print("Enter bx term: ");

        int b = S.nextInt();

        System.out.print("Enter c term: ");

        int c = S.nextInt();

        double d = (b*b) - (4*a*c);
```

```
if(d>0)
{
    System.out.println("Root are REAL and DISTINCT");
    double r1 = (-b + Math.sqrt(d)) / (2 * a);
    double r2 = (-b - Math.sqrt(d)) / (2 * a);
    System.out.println("Root 1 is: " + r1);
    System.out.println("Root 2 is: " + r2);
}
else if(d==0)
{
    System.out.println("Root are REAL and EQUAL");
    double r = -b / (2 * a);
    System.out.println("Root is: " + r);
}
else
{
    System.out.println("Roots are COMPLEX and IMAGINARY");
}
}
```

OUTPUT 1:

Enter ax^2 term: 3

Enter bx term: 4

Enter c term: 1

Root are REAL and DISTINCT

Root 1 is: -0.3333333333333333

Root 2 is: -1.0

OUTPUT 2:

Enter ax² term: 3

Enter bx term: 2

Enter c term: 1

Roots are COMPLEX and IMAGINARY

INFERENCE:

An alternative approach is to create a separate method to calculate and display the roots. This improves code readability and allows the method to be reused in other programs.

6Q) Write a JAVA program to illustrate increment/decrement and bitwise operators.**PROCEDURE:**

Declare integer variables and apply the increment and decrement operators to them. Perform the required bitwise operations on two integer values. Display the result of each operation using println().

and decrement (--) operators using integer variables. It also performs bitwise operations such as AND (&), OR (|), XOR (^), NOT (~), and left shift (<<). The results are displayed using System.out.println().

PROGRAM:

```
public class OperatorDemo
{
    public static void main(String[] args)
    {
        int x = 5;
        System.out.println("Original x: " + x);
        System.out.println("Post-increment (x++): " + (x++));
        System.out.println("Pre-increment (++x): " + (++x));
        System.out.println("Post-decrement (x--): " + (x--));
        System.out.println("Pre-decrement (--x): " + (--x));
    }
}
```

```
System.out.println();  
int a = 5;  
int b = 3;  
System.out.println("(a & b): " + (a & b));  
System.out.println("(a | b): " + (a | b));  
System.out.println("(a ^ b): " + (a ^ b));  
System.out.println("(~a): " + (~a));  
System.out.println("(a << 1): " + (a << 1));  
}  
}
```

OUTPUT:

Original x: 5

Post-increment (x++): 5

Pre-increment (++x): 7

Post-decrement (x--): 7

Pre-decrement (--x): 5

(a & b): 1

(a | b): 7

(a ^ b): 6

(~a): -6

(a << 1): 10

INFERENCE:

The same concepts can also be demonstrated by accepting input values from the user using the Scanner class instead of using fixed values. This helps understand how operators work with different inputs.

7Q) Write a Java program to count and display the number of command-line arguments passed to the program using the args.length property.

PROCEDURE:

Use the args[] array in the main() method to receive command-line values. Find the total number of values using args.length. Display the count on the console.

count the number of command-line arguments passed to the program. The count is displayed using System.out.println().

PROGRAM:

```
class Length
{
    public static void main(String[] args)
    {
        System.out.println("Number of command line arguments = " + args.length);
    }
}
```

OUTPUT:

Number of command line arguments = 5

INFERENCE:

An alternative approach is to use a loop to display each command-line argument along with the total count. This provides more information about the entered arguments.

8Q) Write a Java program to accept a student's Roll Number and Name as command-line arguments and display them on the screen.

PROCEDURE:

Accept the roll number and name through the command-line arguments. Access the required values using args[0] and args[1]. Display the student details using println().

are accepted through the args[] array. The values are accessed using args[0] and args[1] and displayed using System.out.println().

PROGRAM:

```
class Studentdetails
{
    public static void main(String[] args)
    {
        System.out.println("Roll no: " + args[0]);
        System.out.println("Name: " + args[1]);
    }
}
```

OUTPUT:

Roll no: 101

Name: Gayatri

INFERENCE:

The same student details can also be entered using the Scanner class instead of command-line arguments. This allows users to provide input during program execution.

WEEK-2**(Control and iteration Statements, classes and Methods)****1Q) Write a JAVA Program to Illustrate Definition of Class and Creation of Object**

PROCEDURE: Define a Student class with data members and a display method. Create an object of the class inside main() and use it to access the members. Execute the program and display the student details.

PROGRAM:

```
class Student {  
    String name = "Gayatri";  
    int rollNo = 101;  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Roll Number: " + rollNo);  
    }  
}  
  
public class ClassObject {  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.display();  
    }  
}
```

OUTPUT:

Name: Gayatri

Roll Number: 101

INFERENCE:

The program demonstrates how to define a **class** and create an **object** in Java. It shows that a class acts as a blueprint containing data and methods, while an object is an instance of the class used to access its members. This illustrates the basic concept of Object-Oriented Programming (OOP) in Java.

2Q) Write a JAVA Program to Illustrate Selection Statements

PROCEDURE: Read a number and marks from the user using Scanner. Apply if, if-else, else-if, nested-if and switch statements to make the required decisions. Display the corresponding results for the entered values.

PROGRAM:

```
import java.util.Scanner;

public class SelectionStatements {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // if statement

        System.out.print("Enter a number: ");

        int num = sc.nextInt();

        if (num > 0) {

            System.out.println("The number is Positive.");

        }

        // if-else statement

        if (num % 2 == 0) {

            System.out.println("The number is Even.");

        } else {

            System.out.println("The number is Odd.");

        }

        // else-if ladder

        System.out.print("\nEnter marks (0-100): ");

        int marks = sc.nextInt();

        if (marks >= 90) {

            System.out.println("Grade: A");

        } else if (marks >= 75) {

            System.out.println("Grade: B");

        }

    }

}
```

```
} else if (marks >= 50) {  
    System.out.println("Grade: C");  
} else {  
    System.out.println("Grade: Fail");  
}  
  
// Nested if  
System.out.print("\nEnter age: ");  
int age = sc.nextInt();  
if (age >= 18) {  
    if (age >= 21) {  
        System.out.println("Eligible for Driving and Voting.");  
    }  
    else {  
        System.out.println("Eligible for Voting only.");  
    }  
}  
  
else {  
    System.out.println("Not Eligible for Voting.");  
}  
  
// Switch statement  
System.out.print("\nEnter a number (1-7): ");  
int day = sc.nextInt();  
switch(day) {  
    case 1:  
        System.out.println("Sunday");  
        break;  
    case 2:
```

```
        System.out.println("Monday");
        break;
    case 3:
        System.out.println("Tuesday");
        break;
    case 4:
        System.out.println("Wednesday");
        break;
    case 5:
        System.out.println("Thursday");
        break;
    case 6:
        System.out.println("Friday");
        break;
    case 7:
        System.out.println("Saturday");
        break;
    default:
        System.out.println("Invalid Day");
    }
}
```

OUTPUT:

Enter a number: 12

The number is Positive.

The number is Even.

Enter marks (0-100): 85

Grade: B

Enter age: 20

Eligible for Voting only.

Enter a number (1-7): 5

Thursday

INFERENCE:

The selection statements were successfully implemented to control the flow of execution based on the given condition.

3Q) Write a JAVA Program to Illustrate Looping Statements

PROCEDURE: Read the required limit from the user. Apply for, while and do-while loops to generate the numbers up to that limit. Display the sequence produced by each loop.

PROGRAM:

```
import java.util.Scanner;

public class LoopingStatements
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the limit: ");
        int n = sc.nextInt();

        // For Loop
        System.out.println("\nUsing For Loop:");
        for (int i = 1; i <= n; i++)
        {
            System.out.print(i + " ");
        }

        // While Loop
```

```
System.out.println("\n\nUsing While Loop:");
```

```
int i = 1;
```

```
while (i <= n)
```

```
{
```

```
    System.out.print(i + " ");
```

```
    i++;
```

```
}
```

```
// Do-While Loop
```

```
System.out.println("\n\nUsing Do-While Loop:");
```

```
int j = 1;
```

```
do {
```

```
    System.out.print(j + " ");
```

```
    j++;
```

```
} while (j <= n);
```

```
}
```

```
}
```

OUTPUT:

Enter the limit: 5

Using For Loop:

1 2 3 4 5

Using While Loop:

1 2 3 4 5

Using Do-While Loop:

1 2 3 4 5

INFERENCE:

The program demonstrates that looping statements are used to repeat a block of code until the given condition is satisfied, making repetitive tasks easier and reducing code redundancy.

4Q)Write a JAVA Program to Display the Fibonacci Sequence

PROCEDURE: Initialize the first two Fibonacci values as 0 and 1. Repeat the calculation using a for loop, adding the previous two values to obtain the next value. Print each generated term of the sequence.

PROGRAM:

```
public class Fibonacci {  
    public static void main(String[] args) {  
        int n = 8;  
        int first = 0, second = 1, next;  
        System.out.println("Fibonacci Series:");  
        for(int i = 1; i <= n; i++) {  
            System.out.print(first + " ");  
            next = first + second;  
            first = second;  
            second = next;  
        }  
    }  
}
```

OUTPUT:

Fibonacci Series:

0 1 1 2 3 5 8 13

INFERENCE:

Thus, the program successfully generates and displays the Fibonacci sequence using a for loop.

5Q) Write a JAVA Program to Illustrate the Use of Method

PROCEDURE: Define a method that accepts two integer values as parameters. Calculate their sum inside the method and display it. Create an object in main() and call the method with suitable arguments.

PROGRAM:

```
class Addition {  
    void add(int a, int b) {  
        System.out.println("Sum = " + (a + b));  
    }  
}  
  
public class MethodExample {  
    public static void main(String[] args) {  
        Addition obj = new Addition();  
        obj.add(10, 20);  
    }  
}
```

OUTPUT:

Sum = 30

INFERENCE:

The program demonstrates the use of methods in Java. It shows that methods are used to perform specific tasks and can be called whenever needed, making the program more organized, reusable, and easier to understand.

6Q) Write a JAVA Program to Illustrate the Use of Constructor

PROCEDURE: Create a class containing a constructor and a data member. Assign the required initial value inside the constructor. Create an object so that the constructor is invoked automatically, then display the initialized value.

PROGRAM:

```
class Student {  
    String name;
```

```
Student() {  
    name = "Gayatri";  
}  
  
void display() {  
    System.out.println("Student Name: " + name);  
}  
}  
  
public class ConstructorExample {  
    public static void main(String[] args) {  
        Student obj = new Student();  
        obj.display();  
    }  
}
```

OUTPUT:

Student Name: Gayatri

INFERENCE: The program demonstrates the use of constructors in Java. It shows that a constructor is automatically called when an object is created and is used to initialize the object's data. This helps assign initial values to the object's variables.

7Q) Write a JAVA Program to Illustrate the Use of Break, Continue, and Return Statements

PROCEDURE: Create a method that returns the square of a number. Use a loop with continue to skip 5 and break to stop at 8. Call the method and display its returned result.

PROGRAM:

```
class BreakContinueReturn {  
    static int square(int n) {  
        return n * n; // Return statement  
    }  
  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {
```

```
        if (i == 5)
            continue; // Skip number 5
        if (i == 8)
            break;    // Stop loop at 8
        System.out.println(i);
    }
    int result = square(6);
    System.out.println("Square = " + result);
}
```

OUTPUT:

1
2
3
4
6
7

Square = 36

INFERENCE:

The program demonstrates the use of break, continue, and return statements in Java. It shows that break is used to terminate a loop or switch statement, continue skips the current iteration and moves to the next one, and return exits a method by returning control to the calling method.

WEEK-3**(classes and method overloading)**

1Q) Write a Java program to illustrate constructor overloading.

PROCEDURE:

Define two constructors in the same class with different parameter lists. Create separate objects using the available constructors. Display the values initialized by each constructor.

PROGRAM:

```
class Student {  
    Student() {  
        System.out.println("Default Constructor");  
    }  
    Student(String name) {  
        System.out.println("Student Name: " + name);  
    }  
}  
  
public class ConstructorOverloading {  
    public static void main(String args[]) {  
        Student s1 = new Student();  
        Student s2 = new Student("Gayatri");  
    }  
}
```

OUTPUT:

Default Constructor

Student Name: Gayatri

INFERENCE:

The program demonstrates constructor overloading in Java. It shows that a class can have multiple constructors with different parameter lists, allowing objects to be initialized in different ways based on the values provided during object creation.

2Q)Write a Java program to illustrate method overloading.

PROCEDURE:

Define methods with the same name but different numbers of parameters. Create an object and call each overloaded method with suitable arguments. Display the result of every method call.

Compile and execute the program.

PROGRAM:

```
class Addition {  
    void add(int a, int b) {  
        System.out.println("Sum = " + (a + b));  
    }  
    void add(int a, int b, int c) {  
        System.out.println("Sum = " + (a + b + c));  
    }  
}  
  
public class MethodOverloading {  
    public static void main(String args[]) {  
        Addition obj = new Addition();  
        obj.add(10, 30);  
        obj.add(10, 30, 30);  
    }  
}
```

OUTPUT:

Sum = 40

Sum = 70

INFERENCE:

The program demonstrates method overloading in Java. It shows that multiple methods can have the same name but different parameter lists, allowing the same method name to perform different operations based on the arguments passed.

3Q)Write a Java program to illustrate parameter passing.**PROCEDURE:**

Define a method that accepts the required parameters. Pass values to the method from main() through an object. Display the values received by the method.

PROGRAM:

```
class Display {  
    void show(String name, int age) {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}  
  
public class ParameterPassing {  
    public static void main(String args[]) {  
        Display obj = new Display();  
        obj.show("Gayatri", 20);  
    }  
}
```

OUTPUT:

Name: Gayatri

Age: 20

INFERENCE:

It shows how values are passed to methods through parameters, allowing methods to process data and perform operations using the arguments provided during the method call.

4Q) Write a Java program to illustrate one-dimensional and two-dimensional arrays.

PROCEDURE:

Create and initialize a one-dimensional array and display its elements. Then create a two-dimensional array using rows and columns. Read or assign the elements and print the complete array.

PROGRAM:

```
import java.util.Scanner;

public class ArraysDemo
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        // One-Dimensional Array

        System.out.print("Enter the size of 1D array: ");

        int n = sc.nextInt();

        int a[] = new int[n];

        System.out.println("Enter the elements of 1D array:");

        for (int i = 0; i < n; i++)
        {
            a[i] = sc.nextInt();
        }

        System.out.println("\nElements of 1D Array:");

        for (int i = 0; i < n; i++) {
            System.out.print(a[i] + " ");
        }

        // Two-Dimensional Array

        System.out.print("\n\nEnter number of rows: ");

        int rows = sc.nextInt();
```

```
System.out.print("Enter number of columns: ");
int cols = sc.nextInt();
int b[][] = new int[rows][cols];
System.out.println("Enter the elements of 2D array:");
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        b[i][j] = sc.nextInt();
    }
}
System.out.println("\nElements of 2D Array:");
for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        System.out.print(b[i][j] + " ");
    }
    System.out.println();
}
}
```

OUTPUT:

Enter the size of 1D array: 5

Enter the elements of 1D array:

10

20

30

40

50

Elements of 1D Array:

10 20 30 40 50

Enter number of rows: 2

Enter number of columns: 3

Enter the elements of 2D array:

7 8 9

4 5 6

Elements of 2D Array:

7 8 9

4 5 6

INFERENCE:

The program demonstrates the use of one-dimensional and two-dimensional arrays in Java. It shows that a one-dimensional array is used to store a list of elements, while a two-dimensional array is used to store data in rows and columns.

5Q) Write a Java program to sort numbers in ascending and descending order.

PROCEDURE:

Read the numbers into an array and arrange them by comparing pairs of elements. Display the elements in the required order after sorting. Show the resulting sorted values on the console.

PROGRAM:

```
import java.util.Scanner;

public class AscendingOrder
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the number of elements: ");

        int n = sc.nextInt();
```

```
int arr[] = new int[n];
System.out.println("Enter the array elements:");
for (int i = 0; i < n; i++)
{
    arr[i] = sc.nextInt();
}
// Sorting in Ascending Order
for (int i = 0; i < n - 1; i++)
{
    for (int j = i + 1; j < n; j++)
    {
        if (arr[i] > arr[j])
        {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
}
System.out.println("Array in Ascending Order:");
for (int i = 0; i < n; i++) {
    System.out.print(arr[i] + " ");
}
}
```

OUTPUT:

Enter the number of elements: 5

Enter the array elements:

45

12

78

23

10

Array in Ascending Order:

10 12 23 45 78

INFERENCE:

The program demonstrates sorting of numbers in Java. It shows how numbers can be arranged in both ascending and descending order using sorting techniques, making the data easier to organize and analyze.

6Q) Write a Java program to perform Linear Search.

PROCEDURE:

Store the input values in an array and read the element to be searched. Compare the search key with each array element in sequence. Display its position when found; otherwise display that it is not present.

PROGRAM:

```
import java.util.Scanner;

public class LinearSearch
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the number of elements: ");

        int n = sc.nextInt();

        int arr[] = new int[n];

        System.out.println("Enter the array elements:");
```

```
for (int i = 0; i < n; i++)
{
    arr[i] = sc.nextInt();
}

System.out.print("Enter the element to search: ");
int key = sc.nextInt();
boolean found = false;
for (int i = 0; i < n; i++)
{
    if (arr[i] == key)
    {
        System.out.println("Element found at position " + (i + 1));
        found = true;
        break;
    }
}

if (!found) {
    System.out.println("Element not found.");
}
}
```

OUTPUT:

Enter the number of elements: 5

Enter the array elements:

10

20

30

40

50

Enter the element to search: 30

Element found at position 3

Enter the number of elements: 5

Enter the array elements:

10

20

30

40

50

Enter the element to search: 25

Element not found.

INFERENCE:

The program demonstrates the Linear Search technique in Java. It shows how an element is searched by comparing it with each element in the array one by one until the required element is found or the end of the array is reached.

7Q)Write a Java program to add two 2D arrays.

PROCEDURE:

Read the size and elements of two matrices having the same dimensions. Add the corresponding elements of the two matrices and store the sums in a third matrix. Display the resulting matrix.

PROGRAM:

```
import java.util.Scanner;

public class ArrayAddition
{
    public static void main(String[] args)
    {
```

```
Scanner sc = new Scanner(System.in);

System.out.print("Enter the number of rows: ");

int rows = sc.nextInt();

System.out.print("Enter the number of columns: ");

int cols = sc.nextInt();

int a[][] = new int[rows][cols];

int b[][] = new int[rows][cols];

int sum[][] = new int[rows][cols];

// Input first array

System.out.println("Enter the elements of First Array:");

for (int i = 0; i < rows; i++)

{

    for (int j = 0; j < cols; j++)

    {

        a[i][j] = sc.nextInt();

    }

}

// Input second array

System.out.println("Enter the elements of Second Array:");

for (int i = 0; i < rows; i++)

{

    for (int j = 0; j < cols; j++)

    {

        b[i][j] = sc.nextInt();

    }

}

// Addition of arrays
```

```
for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        sum[i][j] = a[i][j] + b[i][j];
    }
}

// Display result

System.out.println("\nSum of Two Arrays:");

for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        System.out.print(sum[i][j] + " ");
    }

    System.out.println();
}
}
```

OUTPUT:

Enter the number of rows: 2

Enter the number of columns: 3

Enter the elements of First Array:

1 2 3

4 5 6

Enter the elements of Second Array:

7 8 9

10 11 12

Sum of Two Arrays:

8 10 12

14 16 18

INFERENCE:

The program demonstrates the addition of two two-dimensional arrays in Java. It shows that corresponding elements of both arrays are added together to produce a new array containing the sum of the elements.

8Q) Write a Java program to pass an array as a parameter to a method.

PROCEDURE: Create an integer array and fill it with values from the user. Pass the array to a separate method as an argument. Traverse the array inside the method and display its elements.

PROGRAM:

```
import java.util.Scanner;

public class ArrayParameter
{
    // Method to display array elements
    static void displayArray(int arr[])
    {
        System.out.println("\nArray Elements are:");
        for (int i = 0; i < arr.length; i++)
        {
            System.out.print(arr[i] + " ");
        }
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the size of the array: ");
```

```
int n = sc.nextInt();  
int arr[] = new int[n];  
System.out.println("Enter the array elements:");  
for (int i = 0; i < n; i++) {  
    arr[i] = sc.nextInt();  
}  
// Passing array to method  
displayArray(arr);  
}  
}
```

OUTPUT:

Enter the size of the array: 5

Enter the array elements:

10

20

35

45

50

Array Elements are:

10 20 35 45 50

INFERENCE:

The program demonstrates passing an array as a parameter to a method in Java. It shows that an entire array can be passed to a method, allowing the method to process or manipulate all its elements efficiently.

WEEK-4**(Array,String,Static Members,String Buffer)****1Q)Write a JAVA Program to Check String Palindrome Property****PROCEDURE:**

Read a string from the user and construct its reverse using a loop. Compare the original and reversed strings with equals(). Display whether the given string is a palindrome.

PROGRAM:

```
import java.util.Scanner;

public class PalindromeString
{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a String: ");

        String str = sc.nextLine();

        String rev = "";

        for (int i = str.length() - 1; i >= 0; i--)
        {
            rev = rev + str.charAt(i);
        }

        if (str.equals(rev))
            System.out.println("The String is a Palindrome.");
        else
            System.out.println("The String is Not a Palindrome.");

        sc.close();
    }
}
```

OUTPUT :

Enter a String: madam

The String is a Palindrome.

Enter a String: hello

The String is Not a Palindrome.

INFERENCE:

the program successfully checks whether the given string is a palindrome by reversing the string and comparing it with the original string. If both strings are identical, the string is identified as a palindrome; otherwise, it is not. This demonstrates the use of string manipulation and comparison techniques in Java.

2Q)Write a JAVA Program to Illustrate String Handling Methods (charAt(), compareTo(), equals(), indexOf(), join(), replace(), substring(), toUpperCase(), toLowerCase())

PROCEDURE:

Create the required string values and apply each specified String handling method. Use the methods to access, compare, search, join, replace, extract and change the case of strings. Print all returned results.

PROGRAM:

```
public class StringMethodsDemo
{
    public static void main(String args[])
    {
        String s1 = "Java";
        String s2 = "Programming";
        System.out.println("charAt(2): " + s1.charAt(2));
        System.out.println("compareTo(): " + s1.compareTo(s2));
        System.out.println("equals(): " + s1.equals("Java"));
        System.out.println("indexOf('a'): " + s1.indexOf('a'));
        System.out.println("join(): " + String.join("-", s1, s2));
        System.out.println("replace(): " + s1.replace('a', 'o'));
```

```

        System.out.println("substring(): " + s2.substring(3, 8));
        System.out.println("toUpperCase(): " + s1.toUpperCase());
        System.out.println("toLowerCase(): " + s2.toLowerCase());
    }
}

```

OUTPUT:

```

charAt(2): v
compareTo(): -6
equals(): true
indexOf('a'): 1
join(): Java-Programming
replace(): Jovo
substring(): gramm
toUpperCase(): JAVA
toLowerCase(): programming

```

INFERENCE:

These methods help in accessing, comparing, searching, modifying, extracting, joining, and changing the case of strings, making string processing simple and efficient.

3Q)Write a JAVA Program to Illustrate Static Variable, Static Method, and Static Block**PROCEDURE:**

Declare a static variable and initialize it through a static block. Define a static method to display the variable and related message. Call the static method directly from main().

PROGRAM:

```

class StaticDemo
{
    static int a;

```

```
static {  
    a = 100;  
    System.out.println("Static Block Executed");  
}  
  
static void display() {  
    System.out.println("Static Method Called");  
    System.out.println("Value of a = " + a);  
}  
  
public static void main(String args[]) {  
    display();  
} }
```

OUTPUT:

Static Block Executed

Static Method Called

Value of a = 100

INFERENCE:

the program illustrates the use of a static variable, static block, and static method in Java.

4Q)Write a JAVA Program to Sort an Array of Strings**PROCEDURE:**

Create an array containing string values. Compare the strings using compareTo() and interchange them when they are out of order. Print the strings after sorting them alphabetically.

PROGRAM:

```
public class StringSort {  
  
    public static void main(String args[]) {  
        String str[] = {"F", "J", "H", "M", "B"};  
        for (int i = 0; i < str.length - 1; i++) {
```

```
for (int j = i + 1; j < str.length; j++) {  
    if (str[i].compareTo(str[j]) > 0) {  
        String temp = str[i];  
        str[i] = str[j];  
        str[j] = temp;  
    }  
}  
  
System.out.println("Sorted Strings:");  
for (String s : str)  
    System.out.println(s);  
}
```

OUTPUT:

Sorted Strings:

B

F

H

J

M

INFERENCE:

the program sorts the array of strings in alphabetical order.

5Q) JAVA Program to Illustrate Five StringBuffer Handling Methods**PROCEDURE:**

Create a StringBuffer containing initial text. Perform append, insert, replace, delete and reverse operations one after another. Display the StringBuffer after each operation.

PROGRAM:

```
public class StringBufferDemo {  
    public static void main(String args[]) {  
        StringBuffer sb = new StringBuffer("Java");  
        sb.append(" Programming");  
        System.out.println("Append: " + sb);  
        sb.insert(5, "Language ");  
        System.out.println("Insert: " + sb);  
  
        sb.replace(5, 13, "Coding ");  
        System.out.println("Replace: " + sb);  
        sb.delete(5, 12);  
        System.out.println("Delete: " + sb);  
        sb.reverse();  
        System.out.println("Reverse: " + sb);  
    }  
}
```

OUTPUT:

Append: Java Programming

Insert: Java Language Programming

Replace: Java Coding Programming

Delete: Java Programming

Reverse: gnimmargorP avaJ

INFERENCE:

the program successfully checks whether the given string is a palindrome by comparing the original string with its reverse.

week-5**(Inheritance, This & Final Keyword, Abstract class, Interface)****1Q) Write a JAVA Program to Illustrate Inheritance, Member Access, Types of Inheritance, and Forms of Inheritance****PROCEDURE:**

Create the parent Animal class with a member and method, then derive Dog from it. Add Puppy and Cat classes to demonstrate multilevel and hierarchical inheritance. Create objects and access the inherited and class-specific members.

PROGRAM:

```
class Animal {  
    String name = "Animal"  
    void eat() {  
        System.out.println("Animal is Eating");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is Barking");  
    }  
    void display() {  
        System.out.println("Member Variable: " + name);  
        eat();  
        bark();  
    }  
}  
  
class Puppy extends Dog {  
    void weep() {  
        System.out.println("Puppy is Weeping");  
    }  
}
```

```
}  
  
}  
  
class Cat extends Animal {  
    void meow() {  
        System.out.println("Cat says Meow");  
    }  
}  
  
public class InheritanceDemo {  
    public static void main(String args[]) {  
        System.out.println("SINGLE INHERITANCE ");  
        Dog d = new Dog();  
        System.out.println("Parent Variable: " + d.name);  
        d.eat();  
        d.bark();  
        System.out.println("MULTILEVEL INHERITANCE");  
        Puppy p = new Puppy();  
        System.out.println("Parent Variable: " + p.name);  
        p.eat();  
        p.bark();  
        p.weep();  
        System.out.println("HIERARCHICAL INHERITANCE ");  
        Cat c = new Cat();  
        System.out.println("Parent Variable: " + c.name);  
        c.eat();  
        c.meow();  
    } }  
}
```

OUTPUT:

SINGLE INHERITANCE

Parent Variable: Animal

Animal is Eating

Dog is Barking

MULTILEVEL INHERITANCE

Parent Variable: Animal

Animal is Eating

Dog is Barking

Puppy is Weeping

HIERARCHICAL INHERITANCE

Parent Variable: Animal

Animal is Eating

Cat says Meow

INFERENCE:the program successfully illustrates inheritance, member access, single inheritance, multilevel inheritance, and hierarchical inheritance. It also mentions that multiple and hybrid inheritance are achieved in Java using interfaces rather than classes.

2Q)Write a JAVA Program to Illustrate the Use of super Keyword

PROCEDURE:

Define a parent class with a constructor and a method. Extend it with a child class and use super() to invoke the parent constructor. Use super.display() to call the parent method and then display the child method.

PROGRAM:

```
class Super1 {
    Super1() {
        System.out.println("super class Constructor");
    }
    void display() {
        System.out.println("super class Method");
    }
}
```

```
}  
Class Base extends Super1 {  
    Base() {  
        super();  
        System.out.println("Base class Constructor");  
    }  
    void show() {  
        super.display();  
        System.out.println("Base class Method");  
    }  
}  
public class SuperDemo {  
    public static void main(String args[]) {  
        Base b = new Base();  
        b.show();  
    }  
}
```

OUTPUT:

Super class Constructor

Base class Constructor

Super class Method

Base class Method

INFERENCE:

the program illustrates the use of the super keyword to access the parent constructor and parent method.

3Q) Write JAVA Program to Illustrate Method Overriding**PROCEDURE:**

Define a method in the parent class and provide another implementation of the same method in the child class. Create an object of the child class. Invoke the method to demonstrate the overridden version.

PROGRAM:

```
class Vehicle {  
    void run() {  
        System.out.println("Vehicle is Running");  
    }  
}  
  
class Bike extends Vehicle {  
    void run() {  
        System.out.println("Bike is Running");  
    }  
}  
  
public class OverrideDemo {  
    public static void main(String args[]) {  
        Bike b = new Bike();  
        b.run();  
    }  
}
```

OUTPUT:

Bike is Running

INFERENCE:

the program illustrates method overriding, where the child class provides its own implementation of the parent class method.

4Q) Write a JAVA program to illustrate this and final keyword.

PROCEDURE:

Declare the student fields and use this in the constructor to assign the parameter values to the instance variables. Declare the college value as final. Create an object and display all the student details.

PROGRAM:

```
class Student
{
    int id;
    String name;
    final String college = "MVGR Engineering College";
    Student(int id, String name)
    {
        this.id = id;
        this.name = name;
    }
    void display()
    {
        System.out.println("Student ID : " + id);
        System.out.println("Student Name : " + name);
        System.out.println("College : " + college);
    }
}

public class ThisFinalDemo
{
    public static void main(String args[])
    {
        Student s1 = new Student(101, "Gayatri");
```

```
s1.display();  
    }  
}
```

OUTPUT:

Student ID : 101

Student Name : Gayatri

College : MVGR Engineering College

INFERENCE:

The program illustrates the use of the this keyword to refer to the current object's instance variables and avoid ambiguity between local variables and instance variables. It also demonstrates the final keyword by declaring a constant (college) whose value cannot be modified after initialization.

5Q) Write a Java program to illustrate the use of an abstract class.

PROCEDURE: Declare an abstract class with an abstract study() method. Extend the class and provide the implementation of study() in the subclass. Create the subclass object and invoke the implemented method.

PROGRAM:

```
abstract class Student  
{  
    abstract void study();  
}  
  
class Student2 extends Student  
{  
    void study()  
    {  
        System.out.println("Student is stuying");  
    }  
}  
  
public class AbstractDemo
```

```

{
    public static void main(String args[])
    {
        Student2 s=new Student2();

        s.study();
    }
}

```

OUTPUT:

Student is studying

INFERENCE:

The program illustrates the use of an **abstract class**. The abstract class Student contains an abstract method study(), which is implemented by the subclass Student2. An abstract class cannot be instantiated directly, so an object of the subclass is created to access the implemented method.

6Q) Write a Java program to illustrate multiple inheritance using interfaces.

PROCEDURE:

Declare two interfaces with separate methods. Create a class that implements both interfaces and provide definitions for both methods. Create its object and call the two implemented methods.

PROGRAM:

```

interface Printable
{
    void print();
}

interface Showable
{
    void show();
}

class Demo implements Printable,Showable
{

```

```
public void print()
{
    System.out.println("Printing.....");
}

public void show ()
{
    System.out.println("Showing....");
}
}

public class Demo
{
    public static void main(String args[])
    {
        Demo obj=new Demo();
        obj.print();
        obj.show();
    }
}
```

OUTPUT:

Printing

Showing

INFERENCE:

The program illustrates **multiple inheritance using interfaces**. Since Java does not support multiple inheritance with classes, a class can implement multiple interfaces to inherit the behavior of more than one interface.

WEEK-6**(Interface Inheritance&User defined Package)****1Q) Write a Java program to demonstrate Interface Inheritance****PROCEDURE:**

Define a parent interface with a method and extend it through a second interface. Implement the child interface in a class and define all inherited methods. Create an object and invoke the methods.

PROGRAM:

```
package week6;

interface Printable
{
    void print();
}

interface Showable extends Printable
{
    void show();
}

public class InterfaceInheritance implements Showable
{
    public void print() {
        System.out.println("printing...");
    }

    public void show() {
        System.out.println("showing...");
    }

    public static void main(String[] args) {
        InterfaceInheritance obj=new InterfaceInheritance();
        obj.show();
    }
}
```

```

    obj.print();
} }

```

OUTPUT :

showing...

printing...

INFERENCE:

The program demonstrates interface inheritance, where one interface inherits another interface. A class implementing the child interface must provide implementations for all inherited methods.

2Q) Write a Java program to illustrate Random class**PROCEDURE:**

Import java.util.Random and create a Random object. Generate values using the required random methods for different data types. Display each generated value on the console.

PROGRAM:

```

import java.util.Random;

public class RandomDemo {
    public static void main(String[] args)
    {
        Random r=new Random();
        System.out.println("random integers :"+r.nextInt());
        System.out.println("random integers from(0-99) :"+r.nextInt(100));
        System.out.println("Boolean :"+r.nextBoolean());
        System.out.println("double value :"+r.nextDouble(3.45));
        System.out.println("float value :"+r.nextFloat(8));
    }
}

```

OUTPUT:

random integers :237210011

random integers from(0-99) :33

Boolean :true

Double value :2.446975196132429

Float value :3.447711

INFERENCE:

The program illustrates the use of the Random class to generate random numbers of different types. It is commonly used in games, simulations, and testing applications.

3Q) Write a Java program to illustrate StringTokenizer class**PROCEDURE:**

Import StringTokenizer and provide a string containing several words. Create a tokenizer for the string and repeatedly obtain tokens while tokens are available. Display the token count and each token.

PROGRAM:

```
import java.util.StringTokenizer;

public class StringTokenizerDemo {

    public static void main(String[] args) {

        String str="welcome to java programming";

        StringTokenizer s=new StringTokenizer(str);

        System.out.println("intial String:"+str);

        System.out.println("no of tokens:"+s.countTokens());

        while(s.hasMoreTokens())

        {

            System.out.println(s.nextToken());

        }

    }

}
```

OUTPUT:

initial String:welcome to java programming

no of tokens:4

welcome

to

java

programming

INFERENCE:

The program demonstrates the StringTokenizer class, which divides a string into smaller parts called tokens based on a delimiter such as space or comma.

4Q) Write a Java Program that reads a line of integers and displays each integer and their sum (Using StringTokenizer)**PROCEDURE:**

Read a complete line of integers and create a StringTokenizer from it. Convert each token into an integer and add it to a running total. Display every integer and the final sum.

PROGRAM:

```
import java.util.Scanner;

import java.util.StringTokenizer;

public class StringTokenizerSum {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter integers separated by spaces:");

        String input = sc.nextLine();

        StringTokenizer st = new StringTokenizer(input);

        int sum = 0;

        System.out.println("Integers are:");

        while (st.hasMoreTokens()) {
```

```
int num = Integer.parseInt(st.nextToken());  
  
System.out.println(num);  
  
sum += num;  
  
}  
  
System.out.println("Sum = " + sum);  
  
sc.close();  
  
}  
  
}
```

OUTPUT:

Enter integers separated by spaces:

10 20 30 40 50

Integers are:

10

20

30

40

50

Sum = 150

INFERENCE:

The program demonstrates how StringTokenizer can be used to separate integers from a single line of input and perform arithmetic operations such as finding their sum.

5Q) Write a Java program to write data into a file using the FileOutputStream class (Byte Stream)**PROCEDURE:**

Import the required I/O classes and create a FileOutputStream for the target file. Convert the text to bytes and write the bytes into the file. Close the stream after writing is complete.

PROGRAM:

```

import java.io.FileOutputStream;
import java.io.IOException;
public class ByteWriteDemo {
    public static void main(String[] args)throws IOException
    {
        FileOutputStream fout=new FileOutputStream("sample.txt");
        String msg="welcome to java io package";
        fout.write(msg.getBytes());
        fout.close();
        System.out.println("Data written Successfully.");
    }
}

```

OUTPUT:

Data written Successfully.

INFERENCE:

The program demonstrates the use of FileOutputStream to write data into a file using a byte stream. It creates the file if it does not exist and stores the given data successfully.

6Q) Write a Java program to read data from a file using the FileInputStream class (Byte Stream)**PROCEDURE:**

Create a FileInputStream for the required file. Read the contents one byte at a time and convert each byte to a character. Display the characters and close the input stream.

PROGRAM:

```

import java.io.FileInputStream;
import java.io.IOException;
public class ByteReadDemo {

```

```
public static void main(String args[])throws IOException
{
    FileInputStream fin=new FileInputStream("sample.txt");
    int ch;
    while((ch=fin.read())!=-1)
    {
        System.out.print((char)ch);
    }
    fin.close();
}
}
```

OUTPUT:

welcome to java io package

INFERENCE:

The program demonstrates the use of FileInputStream to read data from a file using a byte stream. It retrieves the stored data and displays it on the screen.

WEEK-7

(Files,methods,serialization,deserialization)

1Q)Write a Java program to make frequency count of words in a given text.**PROCEDURE:**

Read a line of text and convert it to a common case for comparison. Split the text into individual words and count each occurrence using the required collection. Display the frequency of each word.

PROGRAM:

```
public class Count {  
    public static void main(String[] args) {  
        Scanner s=new Scanner(System.in);  
        String str=s.next();  
        String words[]=str.split(" ");  
        for(int i=0;i<words.length;i++)  
        {  
            int count=1;  
            if(words[i].equals(" "))  
            {  
                continue;  
            }  
            for(int j=i+1;j<words.length;j++)  
            {  
                if(words[i].equals(words[j]))  
                {  
                    count++;  
                    words[j]=" ";  
                }  
            }  
            System.out.println(words[i]+"=" +count);  
        }  
    }  
}
```

```

    }
    s.close();
} }

```

OUTPUT:

welcome java program

welcome=1

INFERENCE:

The program successfully counts and displays the frequency of each word in the given text using a HashMap. It demonstrates the use of collections and string manipulation in Java.

2Q) Write a Java program that reads a file name from the user, then displays information about whether the file exists, whether the file is readable, whether the file is writable, the type of file and the length of the file in bytes.

PROCEDURE:

Read the file name and create a File object for it. Check whether the file exists and obtain the requested file properties when available. Display the information on the console.

PROGRAM:

```

import java.util.*;
import java.io.File;

public class Files {

    public static void main(String[] args) {

        Scanner s=new Scanner(System.in);

        System.out.println("enter a file:");

        File f=new File(s.nextLine());

        System.out.println("exists:"+f.exists());

        System.out.println("can read:"+f.canRead());

        System.out.println("can write:"+f.canWrite());

        System.out.println("is a file:"+f.isFile());
    }
}

```

```

        System.out.println("length"+f.length()+"bytes");
        s.close();
    }
}

```

OUTPUT:

enter a file:sample.txt

exists:true

can read:true

can write:true

is a file:true

length26bytes

INFERENCE:

The program successfully retrieves and displays important properties of a file such as existence, permissions, type, and size using the File class.

3Q) Write a Java program that reads a file and displays the file on the screen, with a line number before each line.

Procedure:

Read the file name and open it with BufferedReader. Start the line number at one and read the file line by line. Print each line together with its corresponding line number.

Program:

```

import java.io.*;
import java.io.IOException;

public class Filesdemo {
    public static void main(String[] args)throws IOException {
        FileReader f=new FileReader("sample.txt");
        BufferedReader bf=new BufferedReader(f);
        String line;
    }
}

```

```

int n=1;
while((line=bf.readLine())!=null)
{
    System.out.println(n+" "+line);
    n++;
}
bf.close();
}
}

```

OUTPUT:

1 welcome to java io package

INFERENCE:

The program successfully reads the contents of a text file and displays each line with its corresponding line number using BufferedReader.

4Q) Write a Java program that displays the number of characters,lines and words in a text file.

PROCEDURE:

Open the specified text file using the required reader. Read every line while maintaining counters for characters, words and lines. Display the three totals after reaching the end of the file.

PROGRAM:

```

import java.io.*;
import java.io.IOException;
public class Textfile {
    public static void main(String[] args)throws IOException {
        BufferedReader bf=new BufferedReader(new FileReader("sample.txt"));
        String line;
        int lines=0,words=0,characters=0;

```

```
while((line=bf.readLine())!=null)
{
    lines++;
    characters+=line.length();
    String[] w=line.split(" ");
    words+=w.length;
}
bf.close();
System.out.println("lines =" +lines);
System.out.println("words =" +words);
System.out.println("characters="+characters);
}
}
```

OUTPUT:

lines =1

words =5

characters=26

INFERENCE:

The program successfully counts the total number of characters, lines, and words present in a text file. It demonstrates file handling and text processing techniques using Java I/O classes.

5Q) Write a Java Program to Illustrate Serialization**PROCEDURE:**

Create a Serializable class with the required data members. Create and initialize an object, then use FileOutputStream and ObjectOutputStream to write the object to a file. Close the streams after serialization.

PROGRAM:

```
import java.io.*;
```

```
class Student implements Serializable {  
    int id;  
    String name;  
    Student(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}  
  
class SerializeDemo {  
    public static void main(String args[]) throws Exception {  
        Student s = new Student(101, "Sameena");  
        FileOutputStream f = new FileOutputStream("student.txt");  
        ObjectOutputStream o = new ObjectOutputStream(f);  
        o.writeObject(s);  
        o.close();  
        f.close();  
        System.out.println("Object serialized successfully");  
    }  
}
```

OUTPUT:

Object serialized successfully.

INFERENCE:

The program successfully stores the object's data into a file using serialization. It demonstrates how Java converts an object into a byte stream for permanent storage or transmission.

6Q)Write a Java Program to Illustrate Deserialization**PROCEDURE:**

Open the serialized file using FileInputStream and ObjectInputStream. Read the stored object and convert it back to its original class type. Display the recovered object data and close the streams.

PROGRAM:

```
import java.io.*;

public class DeserializationDemo {

    public static void main(String[] args) {

        try {

            FileInputStream fis = new FileInputStream("student.ser");

            ObjectInputStream ois = new ObjectInputStream(fis);

            Student s = (Student) ois.readObject();

            System.out.println("ID: " + s.id);

            System.out.println("Name: " + s.name);

            ois.close();

            fis.close();

        } catch (IOException | ClassNotFoundException e) {

            e.printStackTrace();

        }

    }

}
```

OUTPUT:

ID:101

NAME:Gayatri

INFERENCE:

The program successfully retrieves the serialized object from the file using deserialization. It demonstrates how Java reconstructs an object from a byte stream and restores its original state.

WEEK-8**(Arraylist,iterators)**

1Q)write a java program to demonstrate arraylist.

PROCEDURE:

Import ArrayList and create a list to hold the required elements. Add values using add() and access the stored elements using the appropriate list methods. Display the contents of the list.

PROGRAM:

```
import java.util.ArrayList;

public class Arraylist {

    public static void main(String args[])

    {

        ArrayList<String> states= new ArrayList<>();

        states.add("andhra pradesh");

        states.add("tamilnadu");

        states.add("karnataka");

        states.add("uttar pradesh");

        states.add("jammu&kashmir");

        states.add("madhya pradesh");

        states.add("maharastra");

        System.out.println("after insertion the elements in states:"+states);

        System.out.println("the element replaced:"+states.set(3,"rajasthan"));

        System.out.println("the element at position 6"+states.get(6));

        System.out.println("the element removed at position 5"+states.remove(5));

        System.out.println("element replaced: "+states.set(2,"telangana"));

        System.out.println("the size of arraylist:"+states.size());

        System.out.println("is the arraylist empty:"+states.isEmpty());

        System.out.println("after performing all the operations the final arraylist:"+states);

    }
```

```
}
```

OUTPUT:

after insertion the elements in states:[andhra pradesh, tamilnadu, karnataka, uttar pradesh, jammu&kashmir, madhya pradesh, maharastra]

the element replaced:uttar pradesh

the element at position 6: maharastra

the element removed at position 5madhya pradesh

element replaced: karnataka

the size of arraylist:6

is the arraylist empty:false

after performing all the operations the final arraylist:[andhra pradesh, tamilnadu, telangana, rajasthan, jammu&kashmir, maharastra]

INFERENCE:

Thus, the Java program to implement ArrayList was executed successfully. The ArrayList allows elements to be stored dynamically and provides methods to add, access, modify, and remove elements.

2Q) write a java program to illustrate iterator.**PROCEDURE:**

Create an ArrayList and insert several elements. Obtain an Iterator from the list and use hasNext() and next() to traverse it. Display each element during traversal.

PROGRAM:

```
import java.util.ArrayList;
import java.util.Iterator;

public class IteratorDemo {

    public static void main(String args[])
    {
        ArrayList<String> states= new ArrayList<>();
        states.add("andhra pradesh");
```

```
states.add("tamilnadu");
states.add("karnataka");
states.add("uttar pradesh");
states.add("jammu&kashmir");
states.add("madhya pradesh");
states.add("maharastra");
System.out.println("the array list:"+states);
Iterator<String> itr=states.iterator();
while (itr.hasNext()) {
    String n=itr.next();
    System.out.println(n);
    if(n=="tamilnadu")
    {
        System.out.println("the element removed:tamilnadu");
        itr.remove();
    }
}
System.out.println("final arraylist:"+states);
}
```

OUTPUT:

the array list:[andhra pradesh, tamilnadu, karnataka, uttar pradesh, jammu&kashmir, madhya pradesh, maharastra]

andhra pradesh

tamilnadu

the element removed:tamilnadu

karnataka

uttar pradesh

jammu&kashmir

madhya pradesh

maharastra

final arraylist:[andhra pradesh, karnataka, uttar pradesh, jammu&kashmir, madhya pradesh, maharastra]

INFERENCE:

Thus, the Java program to implement Iterator was executed successfully. The Iterator provides a simple and systematic way to traverse the elements of a collection one by one.

3Q) write a java program to demonstrate the use of itearator interface to traverse an array list and remove all even numbers safely during iteration.

PROCEDURE:

Create an ArrayList containing integer values and obtain its Iterator. Traverse the list by checking hasNext() and retrieving each value with next(). Display every element in order.

PROGRAM:

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Iterator;

public class Arraylist2 {

    public static void main(String[] args) {

        ArrayList<Integer> numbers = new ArrayList<>(Arrays.asList(10, 15, 20, 25, 30, 35));

        System.out.println("Original ArrayList: " + numbers);

        Iterator<Integer> itr = numbers.iterator();

        while (itr.hasNext()) {

            Integer num = itr.next();

            if (num % 2 == 0) {

                itr.remove();

            }

        }

    }

}
```

```
}  
    System.out.println("After removing even numbers: " + numbers);  
}  
}
```

OUTPUT:

Original ArrayList: [10, 15, 20, 25, 30, 35]

After removing even numbers: [15, 25, 35]

INFERENCE:

The program demonstrates the use of the Iterator interface to safely traverse and modify an ArrayList. The `remove()` method of the iterator removes even numbers without causing a `ConcurrentModificationException`. Thus, the final ArrayList contains only the odd numbers.

WEEK-9**(Arraylists,generics&reflection API)**

1Q) Write a JAVA program to illustrate Generic Classes.

PROCEDURE:

Create a generic class using the type parameter T and store a value of that type. Create objects of the class with different data types. Use a method to display the stored values.

PROGRAM:

```
class Box<T> {  
    T value;  
  
    Box(T value) {  
        this.value = value;  
    }  
  
    void display() {  
        System.out.println("Value: " + value);  
    }  
}  
  
public class GenericDemo {  
  
    public static void main(String[] args) {  
        Box<Integer> b1 = new Box<>(100);  
        Box<String> b2 = new Box<>("Gayatri");  
        b1.display();  
        b2.display();  
    }  
}
```

OUTPUT:

Value: 100

Value: Gayatri

INFERENCE:

Generic classes are used when we want to give the input of any datatype we want .in the above program we have passed the datatype of integer,string to the same class which would not be possible with the type defined class.

2Q) Write a JAVA program to illustrate Reflection API for retrieving the classes.**PROCEDURE:**

Create a class containing the fields and methods to be examined. Obtain its Class object and use Reflection API methods to retrieve class information. Display the retrieved class name and details.

PROGRAM:

```
class Student {  
    int rollNo;  
    String name;  
    void display() {  
        System.out.println("Student details");  
    }  
}  
  
public class ReflectionClass {  
    public static void main(String[] args) {  
        Class<?> c = Student.class;  
        System.out.println("Class Name: " + c.getName());  
        System.out.println("Simple Name: " + c.getSimpleName());  
    }  
}
```

OUTPUT:

Class Name: Student

Simple Name: Student

INFERENCE:

the Java program to retrieve class information using Reflection API was successfully executed.

In reflection API we can retrieve the class name or method name of the package we are currently working with, it is asking the compiler to return the names or value which are asked through specific methods in reflection API.

3Q) Write a JAVA program to illustrate Reflection API to invoke constructors and obtain the parameters of a constructor.

PROCEDURE:

Create a class with a parameterized constructor. Obtain its Class object through Reflection API and retrieve the available constructors. Examine the constructor parameter information and display it.

PROGRAM:

```
import java.lang.reflect.Constructor;
import java.lang.reflect.Parameter;

class Student {
    String name;
    int age;
    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

public class ReflectionConstructor {
    public static void main(String[] args) throws Exception {
        Class<?> c = Student.class;
        Constructor<?>[] constructors = c.getConstructors();
        for (Constructor<?> con : constructors) {
            System.out.println("Constructor: " + con.getName());
            Parameter[] parameters = con.getParameters();
```

```

        System.out.println("Number of Parameters: "
            + parameters.length);
    for (Parameter p : parameters) {
        System.out.println("Parameter: " + p.getType());
    }
    Student s = (Student) con.newInstance("Sameena", 20);
    System.out.println("Object created successfully");
}
}
}

```

OUTPUT:

Constructor: Student

Number of Parameters: 2

Parameter: class java.lang.String

Parameter: int

Object created successfully

INFERENCE:

Thus, the constructor parameters were obtained and the constructor was successfully invoked using Reflection API.

4Q)Write a JAVA program to demonstrate ArrayList.**PROCEDURE:**

Create an ArrayList and add several elements using add(). Retrieve an element with get(), remove an element with remove(), and obtain the list size using size(). Display the results of these operations.

PROGRAM:

```

import java.util.ArrayList;

public class ArrayListDemo {

```

```

public static void main(String[] args) {
    ArrayList<String> names = new ArrayList<>();
    names.add("Gayatri");
    names.add("kusuma");
    names.add("sneha");
    names.add("sri");
    System.out.println("ArrayList: " + names);
    System.out.println("First Name: " + names.get(0));
    names.remove("Gayatri");
    System.out.println("After Removal: " + names);
    System.out.println("Size: " + names.size());
}
}

```

OUTPUT:

ArrayList: [Gayatri,kusuma,sneha,sri]

First Name: Gayatri

After Removal: [kusuma,sneha, Sri]

Size: 3

Inference:

Thus, the Java program to demonstrate the basic operations of ArrayList was successfully executed.

5Q) Write a JAVA program to create package named p1 and import this package in other classes to get access to members of classes of p1 package.

PROCEDURE:

Create the p1 package and place the Demo class inside it. Define a public method in the package class. Import p1 in another class, create the required object, and call the public method.

PROGRAM:

```
//file 1
package p1;

public class Demo {
    public void display() {
        System.out.println("This is a class inside package p1");
    }
}

//file 2
import p1.*;

public class Test {
    public static void main(String[] args) {
        Demo d = new Demo();
        d.display();
    }
}
```

Output:

This is a class inside package p1

Inference:

the Java program to create and import a package was successfully executed.

6Q)Write a JAVA program to create package named mypack and import it in circle class.

PROCEDURE:

Create the mypack package with the Shape class and define the circle-area method. Import the package into the Circle class outside the package. Create an object and use the imported method to calculate and display the area.

PROGRAM:

```
//file 1  
  
package mypack;  
  
public class Shape {  
    public void area(int r) {  
        double a = 3.14 * r * r;  
        System.out.println("Area of Circle = " + a);  
    }  
}
```

```
//file2  
  
import mypack.*;  
  
public class Circle {  
    public static void main(String[] args) {  
        Shape s = new Shape();  
        s.area(5);  
    }  
}
```

OUTPUT:

Area of Circle = 78.5

INFERENCE:

The Java program to create the mypack package and import it into the Circle class was successfully executed.

WEEK-10**(Exception handling)**

1Q)Write a JAVA program for describing exception handling mechanism.

PROCEDURE:

Place the statement that may cause an arithmetic error inside a try block. Handle the resulting ArithmeticException with a catch block and display a suitable message. Execute the program to observe the exception handling flow.

PROGRAM:

```
public class ExceptionHandlingDemo {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 0;  
        try {  
            int result = a / b;  
            System.out.println("Result = " + result);  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Exception caught: " + e);  
        }  
        finally {  
            System.out.println("Finally block is executed.");  
        }  
        System.out.println("Program continues...");  
    }  
}
```

OUTPUT:

Exception caught: java.lang.ArithmeticException: / by zero

Finally block is executed.

Program continues...

INFERENCE:

The program demonstrates the exception handling mechanism using try, catch, and finally blocks. The `ArithmeticException` is successfully caught and handled.

2Q) Write a Java program using multiple catch clauses to handle four different exceptions.

PROCEDURE:

Place statements capable of producing different exceptions inside the try block. Provide separate catch blocks for the required exception types. Run the program and display the message corresponding to the exception that occurs.

PROGRAM:

```
public class MultipleCatchDemo {  
    public static void main(String[] args) {  
        try {  
            int a = 10;  
            int b = 0;  
            int result = a / b;  
            System.out.println(result);  
        }  
        catch (ArithmeticException e) {  
            System.out.println("ArithmeticException occurred.");  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("ArrayIndexOutOfBoundsException occurred.");  
        }  
        catch (NullPointerException e) {  
            System.out.println("NullPointerException occurred.");  
        }  
    }  
}
```

```

catch (NumberFormatException e) {
    System.out.println("NumberFormatException occurred.");
}
System.out.println("Program completed.");
}
}

```

OUTPUT:

ArithmeticException occurred.

Program completed.

INFERENCE:

The program demonstrates the use of multiple catch clauses. The appropriate catch block is executed depending on the type of exception generated.

3Q)Write a Java program to demonstrate the creation and handling of Java built-in exceptions.

- (i) **ArithmeticException**
- (ii) **NullPointerException**
- (iii) **StringIndexOutOfBoundsException**
- (iv) **FileNotFoundException**
- (v) **NumberFormatException**
- (vi) **ArrayIndexOutOfBoundsException**

PROCEDURE:

Create separate program sections that cause common Java built-in exceptions. Enclose each risky operation in its own try-catch block. Catch the appropriate exception and display a message for each case.

PROGRAM:

```

import java.io.File;

import java.io.FileNotFoundException;

import java.util.Scanner;

public class BuiltInExceptionsDemo {
    \

    public static void main(String[] args) {

```

```
try {  
    int a = 10;  
    int b = 0;  
    System.out.println(a / b);  
}  
catch (ArithmeticException e) {  
    System.out.println("1. ArithmeticException handled.");  
}  
try {  
    String str = null;  
    System.out.println(str.length());  
}  
catch (NullPointerException e) {  
    System.out.println("2. NullPointerException handled.");  
}  
try {  
    String name = "Java";  
    System.out.println(name.charAt(10));  
}  
catch (StringIndexOutOfBoundsException e) {  
    System.out.println("3. StringIndexOutOfBoundsException handled.");  
}  
try {  
    File file = new File("abc.txt");  
    Scanner sc = new Scanner(file);  
    sc.close();  
}  
catch (FileNotFoundException e) {
```

```
        System.out.println("4. FileNotFoundException handled.");
    }
    try {
        int number = Integer.parseInt("Hello");
        System.out.println(number);
    }
    catch (NumberFormatException e) {
        System.out.println("5. NumberFormatException handled.");
    }
    try {
        int arr[] = {10, 20, 30};
        System.out.println(arr[5]);
    }
    catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("6. ArrayIndexOutOfBoundsException handled.");
    }
    System.out.println("All exceptions were handled successfully.");
}
```

OUTPUT:

1. ArithmeticException handled.
2. NullPointerException handled.
3. StringIndexOutOfBoundsException handled.
4. FileNotFoundException handled.
5. NumberFormatException handled.
6. ArrayIndexOutOfBoundsException handled.

All exceptions were handled successfully.

INFERENCE:

The program successfully demonstrates the creation and handling of six Java built-in exceptions using try-catch blocks. Each exception is identified and handled without terminating the program.

WEEK-11**(Exception handling,Multithreading)**

1Q) Write a JAVA program for creation of user defined exception.

PROCEDURE:

Create a new exception class by extending Exception and provide a constructor for its message. Check the required condition in main() and throw the custom exception when the condition is not satisfied. Catch it and display the message.

PROGRAM:

```
class MyException extends Exception
{
    MyException(String message)
    {
        super(message);
    }
}

public class UserException
{
    public static void main(String args[])
    {
        try
        {
            int age = 15;
            if(age < 18)
                throw new MyException("Age must be 18 or above");
            else
                System.out.println("Eligible");
        }
    }
}
```

```
catch(MyException e)
{
    System.out.println(e.getMessage());
}
}
}
```

OUTPUT:

Age must be 18 or above

INFERENCE:

Thus, the Java program for creating and handling a user-defined exception was successfully executed.

2Q)Write a JAVA program to illustrate creation of thread using Runnable interface and extending Thread class.

PROCEDURE:

Create one class implementing Runnable and another class extending Thread, each with a run() method. Create thread objects and start them using start(). Observe the execution of both threads.

PROGRAM:

class MyRunnable implements Runnable

```
{
    public void run()
    {
        System.out.println("Thread created using Runnable interface");
    }
}
```

class MyThread extends Thread

```
{
    public void run()
    {
```

```
        System.out.println("Thread created by extending Thread class");
    }
}

public class ThreadCreation
{
    public static void main(String args[])
    {
        MyRunnable r = new MyRunnable();
        Thread t1 = new Thread(r);
        MyThread t2 = new MyThread();
        t1.start();
        t2.start();
    }
}
```

OUTPUT:

Thread created using Runnable interface

Thread created by extending Thread class

INFERENCE:

Thus, threads were successfully created using both the Runnable interface and Thread class.

3Q)Write a Java program to change and display the name, set and display the priority, and state of the main thread

PROCEDURE:

Obtain the main thread with `currentThread()`. Display its name, priority and state, then change the required properties using the corresponding methods. Print the updated thread information.

PROGRAM:

```
public class MainThread
{
```

```
public static void main(String args[])
{
    Thread t = Thread.currentThread();
    System.out.println("Original Name: " + t.getName());
    System.out.println("Original Priority: " + t.getPriority());
    System.out.println("Original State: " + t.getState());
    t.setName("MyMainThread");
    t.setPriority(8);
    System.out.println("Changed Name: " + t.getName());
    System.out.println("Changed Priority: " + t.getPriority());
    System.out.println("State: " + t.getState());
}
}
```

OUTPUT:

Original Name: main

Original Priority: 5

Original State: RUNNABLE

Changed Name: MyMainThread

Changed Priority: 8

State: RUNNABLE

INFERENCE:

Thus, the name and priority of the main thread were successfully changed and its state was displayed.

4Q)Write a Java program to demonstrate thread methods such as getName(), setName(), getPriority(), setPriority(), sleep(), join(), and isAlive().

PROCEDURE:

Create a Thread subclass and override its run() method. Create a thread object and use getName(), setName(), getPriority() and setPriority() to inspect and modify its properties. Start the thread and display the results.

PROGRAM:

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("Thread is running");
        try
        {
            Thread.sleep(1000);
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

public class ThreadMethods
{
    public static void main(String args[])
    {
        MyThread t = new MyThread();
        System.out.println("Thread Name: " + t.getName());
        t.setName("StudentThread");
        System.out.println("New Thread Name: " + t.getName());
    }
}
```

```
System.out.println("Priority: " + t.getPriority());
t.setPriority(7);
System.out.println("New Priority: " + t.getPriority());
System.out.println("Alive before start: " + t.isAlive());
t.start();
System.out.println("Alive after start: " + t.isAlive());
try
{
    t.join();
}
catch(Exception e)
{
    System.out.println(e);
}
System.out.println("Alive after join: " + t.isAlive());
}
```

OUTPUT:

Thread Name: Thread-0

New Thread Name: StudentThread

Priority: 5

New Priority: 7

Alive before start: false

Thread is running

Alive after start: true

Alive after join: false

INFERENCE:

Thus, the important thread methods such as `getName()`, `setName()`, `getPriority()`, `setPriority()`, `sleep()`, `join()`, and `isAlive()` were successfully demonstrated.

WEEK-12

(Multi-Threading)

1Q) Write a JAVA program that creates three threads. First thread displays "Good Morning" every one second, the second thread displays "Hello" every two seconds and the third thread displays "Welcome" every three seconds.

PROCEDURE:

Create three thread classes and define the required messages in their run() methods. Use sleep() to pause the threads for the specified intervals. Start the threads and observe their displayed messages.

PROGRAM:

```
class GoodMorning extends Thread
```

```
{
    public void run()
    {
        try
        {
            while(true)
            {
                System.out.println("Good Morning");
                Thread.sleep(1000);
            }
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

```
class Hello extends Thread
```

```
{
```

```
public void run()
{
    try
    {
        while(true)
        {
            System.out.println("Hello");
            Thread.sleep(2000);
        }
    }
    catch(Exception e)
    {
        System.out.println(e);
    }
}

class Welcome extends Thread
{
    public void run()
    {
        try
        {
            while(true)
            {
                System.out.println("Welcome");
                Thread.sleep(3000);
            }
        }
    }
}
```

```
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

public class ThreeThreads
{
    public static void main(String args[])
    {
        GoodMorning t1 = new GoodMorning();
        Hello t2 = new Hello();
        Welcome t3 = new Welcome();
        t1.start();
        t2.start();
        t3.start();
    }
}
```

OUTPUT:

Good Morning

Hello

Good Morning

Welcome

Good Morning

Hello

Good Morning

Welcome

...

INFERENCE:

Thus, three threads were successfully created and executed with different time intervals using the sleep() method.

2Q)Write a Java program that illustrates the creation, execution, and behavior of a Daemon Thread.**PROCEDURE:**

Create a thread class whose run() method performs a repeated task. Set the thread as a daemon before starting it. Start the daemon thread and observe that its execution depends on the main thread.

PROGRAM:

```
class DaemonDemo extends Thread {  
    public void run() {  
        while (true) {  
            System.out.println("Daemon thread is running...");  
            try {  
                Thread.sleep(500);  
            } catch (InterruptedException e) {  
                System.out.println(e);  
            }  
        }  
    }  
}  
  
public static void main(String[] args) {  
    DaemonDemo t = new DaemonDemo();  
    t.setDaemon(true);  
    t.start();  
    System.out.println("Main thread is running...");  
    try {  
        Thread.sleep(2000);  
    } catch (InterruptedException e) {  
        System.out.println(e);  
    }  
}
```

```

    }

    System.out.println("Main thread is ending...");
}
}

```

OUTPUT:

Main thread is running...

Daemon thread is running...

Daemon thread is running...

Daemon thread is running...

Daemon thread is running...

Main thread is ending...

INFERENCE:

The program demonstrates that a daemon thread runs in the background and is automatically terminated when all non-daemon threads, such as the main thread, finish execution.

3Q) Write a Java program to illustrate thread synchronization.**PROCEDURE:**

Create a shared resource with a synchronized method. Start multiple threads that use the same resource and call the synchronized method. Execute them to demonstrate controlled access to the shared data.

PROGRAM:

```

class Table {

    synchronized void printTable(int n) {

        for (int i = 1; i <= 5; i++) {

            System.out.println (n * i);

            try {

                Thread.sleep(500);

            } catch (InterruptedException e) {

                System.out.println(e);
            }
        }
    }
}

```

```
    }  
    }  
    }  
}  
class Thread1 extends Thread {  
    Table t;  
    Thread1(Table t) {  
        this.t = t;  
    }  
    public void run() {  
        t.printTable(5);  
    }  
}  
class Thread2 extends Thread {  
    Table t;  
    Thread2(Table t) {  
        this.t = t;  
    }  
    public void run() {  
        t.printTable(10);  
    }  
}  
public class SynchronizationDemo {  
    public static void main(String[] args) {  
        Table obj = new Table();  
        Thread1 t1 = new Thread1(obj);  
        Thread2 t2 = new Thread2(obj);  
        t1.start();
```

```
t2.start();  
}  
}
```

OUTPUT:

```
5  
10  
15  
20  
25  
10  
20  
30  
40  
50
```

INFERENCE:

The program illustrates **thread synchronization** using the synchronized keyword. It ensures that only one thread can access the synchronized method at a time, thereby preventing conflicts when multiple threads share the same resource.

4Q) Write a JAVA program that correctly implements the Producer Consumer problem using the concept of inter-thread communication.

PROCEDURE:

Create a shared buffer that holds one item at a time. Make the producer wait when the buffer is full and make the consumer wait when it is empty, using synchronization with wait() and notify(). Start both threads and display the produced and consumed items.

PROGRAM:

```
class SharedData {  
    int item;
```

```
boolean available = false;

synchronized void produce(int item) {
    while (available) {
        try {
            wait();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
    this.item = item;
    available = true;
    System.out.println("Produced: " + item);
    notify();
}

synchronized void consume() {
    while (!available) {
        try {
            wait();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
    System.out.println("Consumed: " + item);
    available = false;
    notify();
}
}
```

```
class Producer extends Thread {  
    SharedData data;  
    Producer(SharedData data) {  
        this.data = data;  
    }  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            data.produce(i);  
        }  
    }  
}  
  
class Consumer extends Thread {  
    SharedData data;  
    Consumer(SharedData data) {  
        this.data = data;  
    }  
    public void run() {  
        for (int i = 1; i <= 5; i++) {  
            data.consume();  
        }  
    }  
}  
  
public class ProducerConsumer {  
    public static void main(String[] args) {  
        SharedData data = new SharedData();  
        Producer p = new Producer(data);  
        Consumer c = new Consumer(data);  
    }  
}
```

```
p.start();  
    c.start();  
}  
}
```

OUTPUT:

Produced: 1

Consumed: 1

Produced: 2

Consumed: 2

Produced: 3

Consumed: 3

Produced: 4

Consumed: 4

Produced: 5

Consumed: 5

INFERENCE:

The program successfully implements the Producer Consumer problem using inter-thread communication. The wait() and notify() methods coordinate the producer and consumer threads, while synchronized provides safe access to the shared resource.